

Robust PIBT: Handling Livelock and Deadlock via Push & Swap Extensions

Samantha Oh Jia Jia

Monash University

Abstract

Multi-agent path finding (MAPF) is a fundamental problem where multiple agents have to find paths in a shared environment without collision. This paper focuses on the Priority Inheritance with Backtracking (PIBT) algorithm, a rule-based coordination method that resolves conflicts between agents through priority inheritance and backtracking to ensure smooth path planning. However, PIBT remains limited by livelock and deadlock situations, where agents either cyclically interfere without making progress or become mutually blocked.

To overcome these limitations, this research introduces Push-and-Swap primitives to handle deadlock situations that occur when PIBT produces no valid movement plan for an agent. The Push-and-Swap mechanism enables agents to exchange positions under defined priority rules, improving local coordination and recovery efficiency. Furthermore, this paper presents a matrix-based push-counter mechanism for detecting livelocks. Once detected, livelocks are converted into deadlocks, which can then be resolved through fixed priority assignments or swap operations. These extensions enhance PIBT's robustness and adaptability, particularly in dense or dynamic environments.

Code — https://github.com/Samanthaaaa0/pathfinding_for_games.git

Introduction

Multi-agent pathfinding (MAPF) is an important problem for multi-agent planning that aims at computing collision-free paths for a group of agents from their starting points to their goal points. Similar to the shortest paths problem, it aims to minimise the sum of costs, the total number of time steps taken by all agents to reach their goals. Real-world applications include automated warehouse systems, office robots, and game characters in video games. (Stern et al. 2019a)

In automated warehouses, such as those operated by Amazon Robotics, One classic MAPF example is Amazon Robotics, which assigns agents to pick up and deliver packages while they move in narrow corridors. Agents

must avoid blocking each other's paths or passing in restricted spaces and complete their tasks efficiently to maximise throughput.

There are several algorithms to solve the MAPF problem. For example, Conflict-Based Search (CBS) (Sharon et al. 2015) is an optimal search algorithm that resolves conflicts between agents through a hierarchical search. While CBS guarantees optimality and completeness, its performance degrades rapidly as the number of agents increases. Enhanced CBS (ECBS) (Barer et al. 2021) offers a bounded suboptimal alternative that prioritises computational speed over optimality, ensuring that solution costs remain within a predefined suboptimality bound of the optimal cost.

Among existing MAPF algorithms, Priority Inheritance with Backtracking (PIBT) (Okumura et al. 2022) has caught attention for its rule-based coordination mechanism. PIBT allows agents to make local decisions based on dynamically assigned priorities related to their distances from current locations to goals, resolving conflicts through priority inheritance and backtracking without involving other agents. The mechanism gives PIBT high scalability and adaptability for dynamic environments. However, PIBT can suffer from deadlock and livelock situations. A deadlock occurs when all agents' best moves, according to PIBT's policy, are to remain at their current positions, resulting in no further progress. A livelock occurs when agents continuously move back and forth. While they are in motion, they still fail to make any meaningful progress.

Therefore, to address these coordination limitations, this paper proposes extensions to PIBT that enhance its robustness and recovery in dense environments. One of these extensions introduces push-and-swap primitives; in this approach, a swap operation allows two or more agents to exchange positions when the original PIBT is trapped in a deadlock situation. This will allow PIBT to continue to make further progress.

To handle livelock scenarios, we identify specific cyclic interaction patterns that standard PIBT fails to detect or resolve. Our approach monitors the number of agents involved in mutual pushing behaviour. Upon detecting a livelock, one possible strategy is to convert the livelock into a deadlock by fixing the priorities of agents already located at their goals. Once transformed, the situation can then be resolved through PIBT's existing mechanisms and, if necessary, by applying

the swap operation.

Background

MAPF Definition

Multi-Agent Path Finding (MAPF) is a problem that involves planning collision-free paths for multiple agents operating concurrently within a shared environment. Formally, the input to a MAPF problem with n agents is defined as a tuple $(\mathcal{G}, S, T)$. The shared environment is modeled as an undirected graph $\mathcal{G} = (V, E)$, where $V = \{v_1, v_2, \dots, v_k\}$ is a set of vertex, and $E = \{e_1, e_2, \dots, e_m\}$ is a set of edges in the graph, which represents the connection between two vertices. $S : \{s_1, s_2, \dots, s_n\} \rightarrow V$ maps each agent to its unique starting vertex, and $G : \{g_1, g_2, \dots, g_n\} \rightarrow V$ maps each agent to its unique goal vertex (Stern et al. 2019a).

The environment is modelled as a discrete-time system, where at each timestep, an agent may either remain in its current vertex or move to an adjacent vertex. Any agent a_i will occupy one vertex v_i^t at any time given time step t , or (v_i^t, t) for simplicity. A plan π_i for an agent a_i is a list of vertex, time pair, $\pi_i = [(v_i^0, 0), (v_i^1, 1), \dots, (v_i^k, k)]$, where k is the cost of π_i . The MAPF problem is tasked to compute a plan for each agent a_i to traverse from their starting vertex s_i to goal vertex g_i , without colliding to other agents.

MAPF problem considers two types of collision: *vertex collisions* and *edge collisions*. A vertex collision occurs when two agents occupy the same vertex at the same timestep, i.e. agent a_i has $(v_i^{t_i}, t_i)$ and a_j has $(v_j^{t_j}, t_j)$ in their plan, where $v_i^{t_i} = v_j^{t_j}$ and $t_i = t_j$. An edge collision occurs when two agents traverse the same edge in opposite directions during the same timestep, i.e. agent a_i has $[(v_i^{t_i}, t_i), (v_i^{t_i+1}, t_i+1)]$ and a_j has $[(v_j^{t_j}, t_j), (v_j^{t_j+1}, t_j+1)]$ in their plan, where $v_i^{t_i} = v_j^{t_j+1}$, $v_j^{t_j} = v_i^{t_i+1}$ and $t_i = t_j$.

Two commonly used objective metrics in MAPF are the Sum of Costs (SoC) and the makespan. The SoC is defined as the sum of the path lengths for all agents, representing the total travel effort across the system. The makespan, measures the earliest timestep at which all agents have reached at their target vertices. We will focus on SoC as objective in this paper.

MAPF Algorithms

MAPF solvers are generally divided into two main categories: centralised search-based methods and decentralised (or prioritised) planning approaches.

Centralised approaches compute paths for all agents simultaneously using global coordination. Examples include Conflict-Based Search (CBS) (Sharon et al. 2015) and its suboptimal variant Enhanced CBS (ECBS) (Barer et al. 2021), which resolve agent conflicts through constraint-based hierarchical searches. While these methods guarantee completeness and optimality (or bounded suboptimality in the case of ECBS), they are computationally expensive and do not scale well to large numbers of agents or complex environments.

In contrast, decentralised approaches rely on local coordination, where each agent plans its path independently based

on assigned priorities. Algorithms such as Prioritised Planning and Windowed Hierarchical Cooperative A (WHCA) (Silver 2021) are representative of this category. In prioritized planning, agents plan sequentially according to their priorities, with higher-priority agents treated as dynamic obstacles by lower-priority ones. While simple and computationally efficient, this approach is incomplete, as lower-priority agents may fail to find feasible paths in congested environments.

Despite their scalability and suitability for dynamic settings, decentralised methods often encounter deadlock and livelock, where agents block each other or move cyclically without progress. These limitations motivate the development of algorithms such as Priority Inheritance with Backtracking (PIBT), which introduces dynamic prioritisation and local coordination mechanisms to improve decentralised pathfinding performance.

PIBT

PIBT (Okumura et al. 2022) is a rule-based, suboptimal algorithm designed to solve MAPF efficiently through one-time step lookahead iteration. PIBT starts by assigning a unique priority to each agent, and plans the agent's next move in the order of assigned priority; lower priority agent will treat higher priority agent as a hard obstacle at each time step. The core mechanism of PIBT is priority inheritance, i.e. when a high-priority agent's intended move is blocked by a lower-priority agent, the lower-priority agent temporarily inherits the higher priority and attempts to move to clear the conflict. This process effectively prevents priority inversion problems, which is a common issue in many priority-based systems when a high-priority task is blocked by a low-priority task, when there is a third mid-priority task in the system.

In addition, PIBT incorporates a backtracking mechanism to enhance coordination robustness. After attempting to move, if an agent's action fails (i.e., the chosen move is invalid or leads to a conflict), the algorithm backtracks and explores an alternative movement option for that agent. The combination of priority inheritance and backtracking enables PIBT to achieve high scalability when the problem is dense or large.

Although PIBT provides a reachability guarantee under a biconnected graph. It can still face global coordination challenges due to its local planning nature. In particular, PIBT can suffer from deadlocks, where highest priority agent can push the low-priority agent into a dead end, and fails to make any further progress. And also livelocks, which are agents that continue to move repeatedly without making meaningful progress. PIBT guarantees that all agents will eventually arrive to their current goal in lifelong MAPF scenarios (where agents repeatedly receive new tasks), but in one-shot MAPF, where each agent has a single start and goal, this guarantee may not always be upheld. Agents may become trapped in repetitive movement cycles, which violates the completeness in practice.

The local coordination and priority-based scheme of PIBT is highly scalable; however, existing research has not addressed the way to handle the resulting deadlocks and live-

locks in dense scenarios except for LACAM*. Thus, to minimise the gap in decentralised coordination robustness and livelock prevention, this research focuses on improving coordination primitives and recovery approaches.

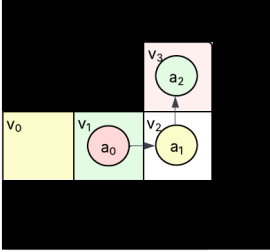
The concept of PIBT with deadlock and livelock resolution is showed in Algorithm 1.

Deadlocks and Livelocks in PIBT

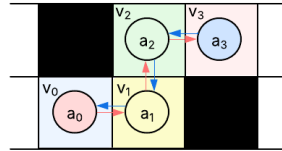
In MAPF, a deadlock occurs when two or more agents block each other's progress indefinitely. On the other hand, a livelock is a dynamic situation where agents keep moving as they are pushing or avoiding each other, but none of them carve out any notable progress. Figure 1 shows the deadlock and livelock examples.

In PIBT, these situations may occur from its local decision-making feature. Each agent plans its movement based on its neighbours and local priority relationships, without global foresight. Although PIBT allows agents to push one another indirectly through chains of priority inheritance, this mechanism can still lead to cyclic dependencies. For example, a group of agents may repeatedly push each other in a loop, creating a livelock. Similarly, deadlocks may occur when a group of agents occupy mutually and block each other, which lead to getting stuck. As PIBT only performs local coordination, it lacks the ability to detect and resolve such global standoffs effectively.

Although PIBT effectively prevents small-scale local collisions, the lack of identifying and handling global deadlocks and livelocks represents a notable research gap. Therefore, to fix this gap, we need a mechanism that can both detect such settings and restore progress without sacrificing PIBT's decentralised feature.



(a) Dead lock: a_0 's goal vertex is v_3 , a_1 's goal vertex is v_1 and a_2 's goal vertex is v_3



(b) Live lock: a_0 's goal vertex is v_3 , a_1 's goal vertex is v_1 , a_2 's goal vertex is v_2 and a_3 's goal vertex is v_0

Figure 1: Partial map of where dead lock and live lock could occur

Push and Swap Algorithm

The Push and Swap algorithm (Luna and Bekris 2011) was designed as a complete approach for MAPF that allows agents to locally resolve conflicts by two simple operations, which are push and swap. These operations allow agents to locally rearrange their positions without centralised coordination.

Algorithm 1: PIBT + Extensions

Require: grid $\mathcal{G}, S : [s_1, \dots, s_n], G : [g_1, \dots, g_n]$

Ensure: paths $[\pi_1, \dots, \pi_n]$

```

1: for each timestep  $t = 1, 2, \dots$  until termination do
2:   if  $P[k][i] > \theta$  then
3:     RESOLVELIVELOCK( $a_i$ )
4:   end if
5:   if  $\pi_i[t] \neq g_i$  then
6:      $p_i \leftarrow p_i + 1$ 
7:   end if
8:   Sort  $A$  in decreasing order of  $p$ 
9:   for each  $a_i \in A$  do
10:    if  $\pi_i[t+1] = \perp$  then
11:      PIBT( $a_i, \perp$ )
12:    end if
13:  end for
14: end for

15: function PIBT( $a_i, r$ )
16:   if  $a_i$  in livelock then
17:     RESOLVELIVELOCK( $a_i$ )
18:   end if
19:    $C \leftarrow N(\pi_i[t]) \cup \{\pi_i[t]\}$   $\triangleright$  Candidate vertices
20:   Sort  $C$  by  $dist(v, g_i)$   $\triangleright$  Closest to goal first
21:    $S \leftarrow \emptyset$   $\triangleright$  Swap candidates
22:   for  $v \in C$  do
23:     if  $\exists a_k : \pi_k[t+1] = v$  then
24:       continue  $\triangleright$  Vertex conflict
25:     end if
26:     if  $\exists a_k : \pi_k[t] = v \wedge \pi_k[t+1] = \pi_i[t]$  then
27:       continue  $\triangleright$  Edge conflict
28:     end if
29:      $\pi_i[t+1] \leftarrow v$ 
30:     if  $\exists a_k : \pi_k[t] = v \wedge \pi_k[t+1] = \perp$  then
31:        $P[k][i] \leftarrow P[k][i] + 1$   $\triangleright$  Push count
32:       if  $\neg \text{PIBT}(a_k, r)$  then
33:         if  $a_i = r$  then
34:            $S \leftarrow S \cup \{(a_k, v)\}$ 
35:         end if
36:       continue
37:     end if
38:   end for
39:   return True
40: end for
41: if  $a_i = r \wedge S \neq \emptyset$  then
42:   if TRYSWAP( $S$ ) then
43:     return True
44:   end if
45: end if
46:    $\pi_i[t+1] \leftarrow \pi_i[t]$ 
47:   return False
48: end function

```

A push operation occurs when one agent (Agent *B*) finds its path blocked by another agent (Agent *A*). In this case, Agent *B* repeatedly pushes Agent *A* away from the blocking position until a free vertex becomes available, allowing Agent *B* to continue towards its goal. If pushing alone is insufficient, such as when two agents must exchange positions, a swap operation is a nice tool to allow them to exchange their locations safely without making any collision. The procedure proceeds as follows:

1. Identify the closest vertex v with degree ≥ 3 to serve as a temporary exchange location.
2. Use push operations to clear a path to v and ensure its neighboring vertices are unoccupied.
3. Move agents *A* and *B* along the cleared path to v .
4. Exchange the positions of *A* and *B* at v .
5. Restore all displaced agents to their original positions.

Push and Swap relies on local decisions, which are well-suited for integration with PIBT. Its ability to resolve local impasses complements PIBT’s priority-based coordination. However, Push and Swap itself can suffer from inefficiency in dense environments and may require a large number of auxiliary moves to achieve a swap.

In this paper, Push and Swap is treated as an extension to PIBT to address deadlock and livelock limitations. Agents can temporarily exchange their positions when priority-based pushing fails, which could guarantee progress without breaking PIBT’s structure.

LaCAM

Lazy Constraints Addition for MAPF (LaCAM) (Okumura 2023b) is a complete and suboptimal MAPF solver to address large-scale MAPF problems more efficiently. It combines fast and decentralised PIBT method with a centralised search to ensure completeness. LaCAM conducts a systematic search in the joint configuration space, but only generates and enforces constraints to avoid redundant work. At the high level, each search node represents a particular joint configuration and an associated set of collision-avoiding constraints. For each such node, LaCAM’s low-level procedure generates the next configuration by running a PIBT-like assignment under the current constraints. Since PIBT is very fast at proposing next moves but can deadlock, LaCAM’s high-level search wraps this in a search: if PIBT fails under certain constraints, LaCAM backtracks and tries different constraints or paths. This two-level structure—constraints at the low level and search over configurations at the high level—allows LaCAM to escape local minima and guarantee finding a solution if one exists.

In summary, LaCAM wraps PIBT in a global search loop. Vanilla PIBT is completely decentralised and fast per step, but it can’t recover from deadlocks and livelocks. LaCAM inherits PIBT’s decentralised structure for generating candidate moves while enforcing completeness by adding constraints when PIBT gets stuck. As a result, LaCAM sacrifices some decentralised method (as it aims to maintain a global search) to gain completeness.

Building on this, LaCAM* (Okumura 2023a) was later introduced as an anytime extension of LaCAM, converging toward optimality in terms of SoC under consistent heuristic assumptions. LaCAM* starts with an initial feasible (possibly suboptimal) solution and iteratively refines it by reusing constraints and improving cost bounds. This feature allows LaCAM* to adapt to both real-time and offline planning scenarios. Experimental evaluations show that LaCAM* maintains high success rates in large-scale scenarios, demonstrating a balanced trade-off between optimality and scalability.

Methodology

As this research involves algorithmic implementation and computer simulations rather than human subjects, there is no requirement for ethics review concerning human data. The methodology of this paper computes research, exploration, implementation, and experimental evaluation.

Algorithm Design

Choice of problem scope To focus on the effectiveness of coordination extensions and ensure controlled testing, this study is limited to 2D grid-based static environments. Agents are homogeneous (they move at the same speed and occupy one grid cell each) and obstacles are fixed (no dynamic obstacles or environmental changes). Agents remain in place once they reach their goal. Time is discrete and synchronous, and moves are restricted to nearest neighbours or staying in place.

The decision to implement the algorithm in Python was driven by the need for rapid prototyping and ease of debugging. Python’s expressive syntax and ease of modifying code make it well-suited for exploring algorithmic variants (e.g. different livelock thresholds, push and swap behaviour). However, the performance of Python can fall behind compiled languages. Particular concerns include per-step decision loops, distance calculations, and handling push and swap logic. To mitigate this, the implementation will precompute and use efficient data structures. For a faster approach, C++ can be used in a future implementation.

Base PIBT Instead of building a new algorithm from scratch, Priority Inheritance with Backtracking (PIBT) (Okumura et al. 2022) provides a strong base. PIBT’s decentralised coordination allows the focus to remain on improving coordination behaviour rather than path optimality. Its lightweight architecture enables handling of hundreds or thousands of agents efficiently.

Although PIBT handles local conflicts well via priority inheritance and backtracking, livelocks and dead-ends can still occur in dense or constrained grid layouts, especially when agents block one another cyclically. (e.g. PIBT’s reachability guarantee depends on graph assumptions and does not preclude cyclic non-progress).

In the proposed design, PIBT remains the core loop: in each timestep agents attempt moves according to PIBT’s rules; when certain failure or livelock conditions are detected, control will switch over to extension modules (e.g. push and swap logic, priority freezing) to recover from those states.

Detect Deadlock

In MAPF, deadlock occurs when two or more agents are mutually blocking each other's progress, which means that no agent can move without violating the collision constraints, which corresponds to a cyclic dependency. While PIBT's built-in backtracking can resolve many local conflicts, it does not guarantee to resolve all forms of circular dependencies. Formally, a deadlock exists when agents exhaust their available moves and backtracking fails to produce a feasible reordering of priorities.

Instead of waiting multiple timesteps to detect stagnation, the algorithm was made to check priorities at the moment of conflict. Specifically, if all agents blocking the move of agent a have lower priority, this implies that no higher-priority agent can resolve the blockage.

Push and Swap Resolution

To address this, the Push-and-Swap mechanism (Luna and Bekris 2011) was integrated as an external deadlock recovery mechanism. This mechanism is activated once the following conditions are met: (1) The agent's intended move is blocked, and all blocking agents have equal or lower priority, indicating a deadlock; (2) PIBT's backtracking fails to produce a valid move for the agent.

Once a deadlock is detected for agent a_i , the Push-and-Swap module operates by identifying the minimal set of agents involved in the cyclic block. For each agent a_j blocking a_i , include a_j in the deadlock set and repeat the process for any agents blocking a_i until the cycle closes.

$$\text{DeadlockSet}(a_i) = \{a_i\} \cup \bigcup_{b \in \text{blockers}(a_i)} \text{DeadlockSet}(b)$$

and performing one of two key operations:

- Push: Attempting to shift a blocking agent along an alternate path to free up the target cell for a higher-priority agent.
- Swap: Exchanging positions between two adjacent agents when there is no feasible push sequence exists, thereby breaking the circular wait condition. Detailed steps has been discussed in section .

Algorithm 2: Try Swap

```

1: function TRY_SWAP( $a_i, a_j, \pi_{from}, \pi_{to}$ )
2:    $\pi_{temp} \leftarrow \pi_{from}$ 
3:    $D \leftarrow \text{SIMULATE\_PIBT}(a_i, \text{None}) \triangleright$  Simulate PIBT
   to detect additional swap chain
4:   Find candidate vertices  $V_{\text{cand}}$  with high degree
5:   for each  $v$  in  $V_{\text{cand}}$  do
6:     if CORRIDOR_SWAP( $i, \mathcal{S}, v, C_{from}, C_{to}$ ) then
7:       return True
8:     end if
9:   end for
10:  return False
11: end function

```

This approach is inspired by the Push-and-Rotate methodology (De Wilde, Ter Mors, and Witteveen 2014). which

resolves deadlocks by identifying a high-degree vertex and using it as an exchange location. Firstly, detect all agents $a_i, a_2, \dots, a_k$ that are involved in the deadlock cycle by simulating PIBT once a swap is needed. By detecting the closest high-degree vertex v and clearing its neighbours and path to v , the exchange environment has been set up. Move all the involved agents to queue up on v to wait for performing exchange coordination sequentially (each agent performs exchange with the root agent one at a time). However, performance would be significantly degraded when $k > 3$ in grid environments, as only a pair of agents are allowed to swap at one time, while the rest have to wait.

For deadlocks involving multiple agents ($k > 3$), sequential swaps at a single vertex can be inefficient. Instead, a corridor-based swap is implemented, refer to Algorithm 3:

1. Identify a linear path (corridor) $P = \langle v_1, v_2, \dots, v_m \rangle$ adjacent to the deadlock cycle that contains agents involved in the deadlock.
2. Designate a facilitator agent a_f and temporarily move it out of the corridor to create contiguous free space.
3. The remaining agents traverse the corridor sequentially, performing swaps as needed, until all deadlocked agents have passed through.
4. The facilitator agent a_f then returns to its goal, restoring the normal configuration.

This procedure effectively transforms the agent order from

$$\langle a_0, a_1, a_2, \dots, a_k \rangle \rightarrow \langle a_1, a_2, \dots, a_k, a_0 \rangle$$

breaking the circular dependency while preserving PIBT's decentralised operation.

Algorithm 3: Corridor Swap

```

1: function CORRIDOR_SWAP( $a_i, D_i, v$ )
2:   SETUPAREA( $D_i, v$ )  $\triangleright$  Prepare corridor region
3:    $w \leftarrow \text{neigh}(v)$ 
4:   MOVE( $a_i, w$ )  $\triangleright$  Facilitator steps aside
5:   for each  $a_k \in D_i \setminus \{a_i\}$  do
6:     MOVE( $a_k, v$ )
7:      $u \leftarrow N(v) \setminus \{w\}$ 
8:     if  $u$  occupied and  $\neg \text{CLEAR}(u, \{v\})$  then
9:       return False
10:    end if
11:    MOVE( $a_k, u$ )
12:  end for
13:  MOVE( $a_i, v$ )
14:  RESTORE( $D_i$ )
15:  return True
16: end function
17:
18: function SETUPAREA( $D_i, v$ )
19:   CLEARPATH( $v$ )
20:   CLEAR( $v$ )
21:   MOVE_AGENTS( $D_i, v$ )
22: end function

```

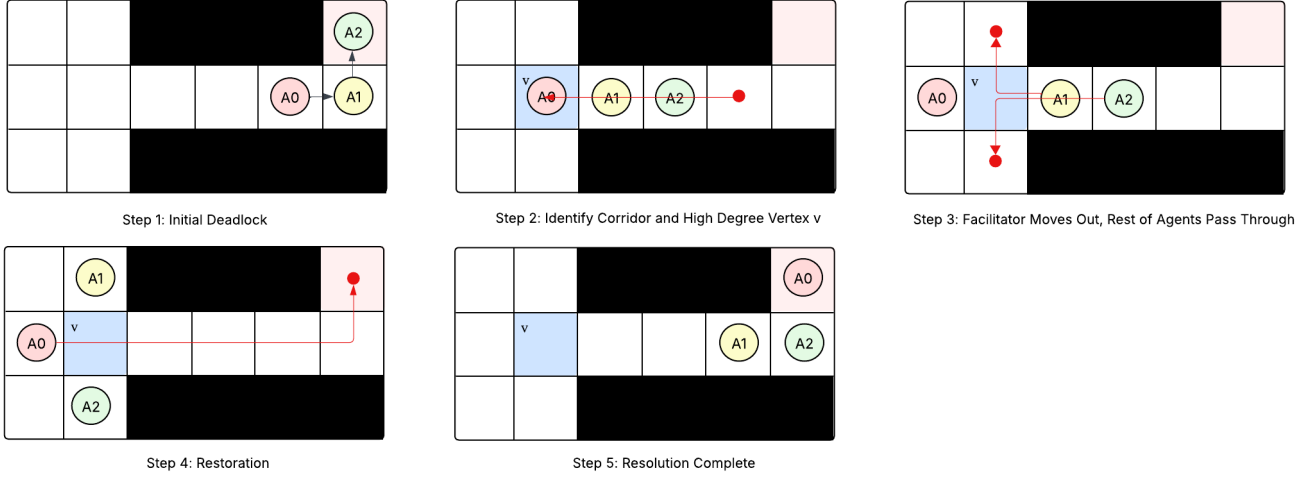


Figure 2: Corridor swap resolution sequence for a three-agent deadlock. **Step 1:** Initial circular dependency among A0, A1, A2. A0 could never reach its goal (pink background) by pushing A1 and A2 at all. **Step 2:** Identify a high-degree vertex v (in blue background) and corridor adjacent to the cycle. **Step 3:** Facilitator agent moves out through the corridor to create contiguous free space. **Step 4:** Remaining agents (A1 and A2) traverse out the corridor sequentially. **Step 5:** Circular dependency resolved, restore involved agents to their original position.

Handle Livelock

Instead of handling livelock as a completely distinct failure class, this research categorise livelock as a case of unresolved cyclic dependencies that can be addressed through existing deadlock recovery mechanisms by converting livelock to deadlock. This design choice leverages PIBT’s established priority inheritance and backtracking features, and it maintains algorithmic consistency by reusing Push-and-Swap, which was developed for deadlock resolution. This reduces conceptual complexity and avoids duplicating logic.

The theory aligns with the concept of the Push-and-Swap algorithm (Luna and Bekris 2011), which Luna and Bekris demonstrated can resolve any solvable MAPF instance by systematically breaking circular dependencies. Additionally, despite its dynamic priority structure and backtracking, PIBT can actually be viewed as a composite of safe “push” operations, even though it does not include a swap primitive. Therefore, integrating swap logic is a logical fit with the design philosophy of PIBT. Therefore, by converting detected livelock to deadlock, the coordination mechanisms of PIBT (i.e. priority inheritance and backtracking) are still able to operate on well-defined conflict states.

Livelock Detection Detecting livelock in MAPF, especially in a decentralised system is always a tricky challenges, as the computer could never read a single agent to process global state information. Therefore I had three strategies:

Sliding Window: By maintaining a fixed-size buffer w that moves every timestep, keeping track of recent agent configurations and checks if agents are making progress or only moving back and forth. Although it is conceptually simple, this approach has high memory overhead, $O(nw)$ where n is the number of agents and w is the window size.

The choice of w is tricky as it may miss oscillations that exceed the window duration.

Distance Table Monitoring: As PIBT itself has a distance table to update priorities, by reusing it to track each agent’s shortest-path distance using Manhattan distance to its goal over time. It flags livelock when distances fail to decrease over T timesteps. This approach is memory-efficient, but at the same time it may produce false positives in congested areas where temporary stalls are normal.

Matrix State: By building a $n \times n$ matrix where n is the number of agents from the start, when PIBT runs, accumulate the counter when any agent get pushed. According to figure 3, Agent 0, which has the highest priority, in order to reach its goal, needs to push Agent 1 and 2 away. Therefore, when the lower-priority agent (i.e. A1 and A2) inherits the higher-priority agent (i.e. A0), it will be recorded on the matrix, i.e. $matrix[A0][A1] + = 1$. If the push counter hits a threshold, let’s say three agents are stuck in a livelock, we can see the three agents has extremely large counts in the matrix.

This implementation adopts the matrix approach as even it requires $O(n^2)$ to build a matrix, but for further access it only needs $O(1)$. It captures global cyclic behavior, not just local agent repetition. Additionally, it avoids false positives from temporary congestion while reliably identifying genuine non-progress patterns.

Livelock Resolution Instead of handling livelock as a completely distinct failure cases, this research addresses livelocks by converting into deadlocks to trigger deadlock recovery mechanisms. The first step in livelock resolution is to freeze the priority ordering of agents currently involved in the detected cycle. Unlike PIBT’s standard dynamic priority updates (where priorities change based on graph dis-

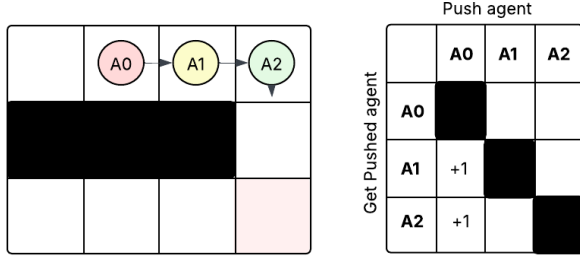


Figure 3: Push Counter Matrix (right) used for detecting livelock in PIBT.

tance), agents in livelock have their priorities fixed at their current values. As a result, the oscillating, fluid system becomes a static dependence structure that can be analysed for deadlocks. Agents that have successfully reached their goals $pos(a_i) = goal(a_i)$ are treated as having assigned maximum priority to prevent displacement which marked as immovable obstacles, exempt from swap operations. This protection ensures that agents which have completed their tasks are not unnecessarily disturbed during livelock resolution. With priorities fixed, it will try to return to the standard PIBT coordination loop before considering it as a deadlock scenario. This rationale is that there's livelocks can potentially be solvable through standard backtracking under fixed priorities. If all agents make measurable progress towards their goals, the livelock is considered resolved and dynamic priorities are restored. Otherwise, if agents remain stalled, the system concludes that a true circular dependency exist and identify it as a deadlock with the Push-and-Swap resolution to break the cycle. However, to maintain goal agent stability, the implementation has to follow a rule: an agent a_i may only trigger swap with agent a_j if

$$P(a_i) > P(a_j) \vee (a_j \in GoalSet \wedge P(a_i) < P(j))$$

where $GoalSet = \{a | pos(a) = goal(a)\}$ and $P(a)$ is the priority value. This allows goal agents to participate in swaps only when needed and the resolution process still respects the priority semantics established by PIBT.

When a swap is required, the corridor swap mechanism 2 is employed to minimise the total displacement. The higher priority goal agent temporarily vacates its position through the corridor, allowing the blocked agent to advance, then returns to its goal location.

Restoration

After the Push-and-Swap phase completes, the restoration function is responsible for returning all involved agents to their final valid configurations to maintain consistency across the system state. Each agent's movements during swapping are recorded as two distinct sequences: the move stack $M[a]$, which records the path from a 's original position to v , and the swap stack $S[a]$, which records the path from v to whichever position it went through. These stacks are later combined to reconstruct the agents' final positions.

To preserve consistency after a swap, where agents exchange positions, the algorithm constructs a rotated move stack, in which each agent inherits the previous agent's motion history. The last agent's path is cyclically assigned to the first agent, ensuring alignment with the new positional order created by the swap.

Next, the function merges the rotated stack of each agent with its corresponding swap stack and removes redundant moves using a trimming routine. The agents are then restored sequentially, beginning with the swap initiator (root agent) followed by the remaining agents in reverse order.

Finally, a new configuration is generated, capturing the restored agent positions at each timestep. This ensures that after the swap, all agents are returned to stable positions without conflicts, preserving the global validity of the MAPF solution.

Algorithm 4: Restoration Function

```

1: function RESTORE( $i, I$ )
2:    $R[a] = []$  for all  $a \in A, config = []$ 
3:   for each agent  $a_k \in I$  do
4:     if  $a_k = i$  then
5:        $R[a_k] \leftarrow M[a_{|I|}]$   $\triangleright$  Rotate last agent's path
        to the first
6:     else
7:        $R[a_k] \leftarrow M[a_{k-1}]$   $\triangleright$  Shift previous agent's
        path
8:     end if
9:   end for
10:  for each  $a \in I$  do
11:     $S[a] += R[a]$ 
12:    Reverse  $R[a]$ 
13:  end for
14:   $O = [i] + [a \in I \setminus \{i\}]^{rev}$   $\triangleright$  Define restoration
    order
15:  for  $t = 1$  to  $\max(|R[a]|)$  over  $a \in O$  do
16:    for  $a \in O$  do
17:      if  $t \leq |R[a]|$  then
18:         $config[a] \leftarrow R[a][t]$ 
19:      end if
20:    end for
21:  end for
22:  return  $config$ 
23: end function

```

Experiment

Experimental Setup

We evaluate our enhanced algorithm (PIBT + Extensions) and compare it with the baseline PIBT on two different benchmarks grid-environments of varying size and agent density. Each experiment we involved the same set of agents, start and goal configurations, and environmental constraints to ensure fairness. For each scenario we measure:

- **Runtime:** the amount of time it takes for all agents reach at their target vertices.

- **Sum of Costs (SoC):** the total number of steps taken by all agents until arrival.
- **Success Rate:** the percentage of runs in which all agents reached their goal without collisions or unresolved deadlocks.

Each algorithm was run on each scenario for 25 random seeds. The experiment is performed on a server AMD EPYC-Milan 3.7GHz, with 128 GB of RAM in total.

Results

Analysis of Outcomes

Our experimental results reveal an important insight about the trade-offs in decentralised MAPF systems. As shown in Figures ?? and ??, the vanilla PIBT already achieves high success rates (90-100%) on these benchmark maps, indicating that deadlock and livelock scenarios are relatively rare in moderately dense environments with this map topology. The proposed extensions introduce a measurable overhead which are runtime overhead, where detection mechanisms and swap coordination have added 5x computational cost at high agent densities, while the SoC still remains comparable, which indicates the path quality is preserved.

Discussion

In this discussion, we proposed deadlock and livelock detection and recovery framework for PIBT within the broader landscape of MAPF algorithms. The primary aim of this research was to enhance the PIBT by integrating livelock detection and efficient deadlock recovery, thereby improving its robustness in densely populated and constrained grid environments. While PIBT effectively handles many local conflicts through priority inheritance and backtracking, it remains vulnerable to livelocks and dead ends.

LaCAM* represents a different paradigm by performing search over joint configurations and leveraging lazy successor generation to handle very large instances. In contrast, our method remains within the PIBT operational framework, and focuses on detecting and resolving livelocks/deadlocks rather than having a centralised search. This ensures the performance remains as efficient as PIBT. While LaCAM prevents deadlock by pattern detection, and avoids the situation when a swap is mandatory to make further progress (Okumura 2023a), our approach differs in that we transform livelocks into deadlocks and reuse the deadlock recovery (push-and-swap) uniformly. This simplifies the detection and resolution pipeline and reduces reliance on global pattern detection heuristics.

Our results suggest that deadlock/livelock recovery mechanisms are most beneficial in scenarios with map with narrow corridors and asymmetric goal distributions. And we had tested on some benchmarks on different number of agent which increase by 5 for each case. (Stern et al. 2019b)

Limitations Although the integration of Push-and-Swap into PIBT plus our livelock-to-deadlock conversion significantly improves robustness and reduces failure cases in empirical testing, the method remains heuristic. It does not offer

a formal guarantee of completeness across all map topologies and agent densities in the same way that LaCAM* do. The prioritisation and decentralised architecture impose implicit constraints on the agent-interaction graph, and the threshold-based detection may fail under rare configurations (e.g., highly symmetric oscillations with no push events). Thus while practically effective, the solver is not theoretically complete.

Empirical Reflection Although the enhanced method did not outperform the baseline in the conducted experiments, this outcome still yields important insights. The extra coordination layer likely introduced unintended delay and complexity. In addition, limited parameter tuning and time-constraints on debugging may have prevented the algorithm from reaching its potential. Nonetheless, the experiments confirm the feasibility of integrating livelock and deadlock mechanisms into a decentralised MAPF solver and provide a foundation for further work.

Conclusion

In this study, we explored how livelock detection and deadlock recovery could be integrated into PIBT for MAPF. Although these extensions aimed to make the algorithm more robust, our experiments showed that it would introduce more overhead than benefits. This outcome might also be influenced by limitations in our implementation, suggesting that further optimisation could change these results. Still, several key lessons emerged: (1) the usefulness of safety mechanisms depends on context and baseline failure rates, (2) the cost of decentralised coordination can sometimes outweigh the impact of rare failures, and (3) future work should explore adaptive activation, enabling recovery only when necessary.

Acknowledgments

I am grateful to Andy Li, Daniel Harabor and Peter Stuckey for supervising.

References

- Barer, M.; Sharon, G.; Stern, R.; and Felner, A. 2021. Sub-optimal Variants of the Conflict-Based Search Algorithm for the Multi-Agent Pathfinding Problem. *Proceedings of the International Symposium on Combinatorial Search*, 5(1): 19–27.
- De Wilde, B.; Ter Mors, A. W.; and Witteveen, C. 2014. Push and rotate: a complete multi-agent pathfinding algorithm. *J. Artif. Int. Res.*, 51(1): 443–492.
- Luna, R.; and Bekris, K. 2011. Push and Swap: Fast Cooperative Path-Finding with Completeness Guarantees. 294–300.
- Okumura, K. 2023a. Improving LaCAM for Scalable Eventually Optimal Multi-Agent Pathfinding. arXiv:2305.03632.
- Okumura, K. 2023b. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*.

- Okumura, K.; Machida, M.; Défago, X.; and Tamura, Y. 2022. Priority Inheritance with Backtracking for Iterative Multi-agent Path Finding. *Artificial Intelligence*, 103752.
- Sharon, G.; Stern, R.; Felner, A.; and Sturtevant, N. R. 2015. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219: 40–66.
- Silver, D. 2021. Cooperative Pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 1(1): 117–122.
- Stern, R.; Sturtevant, N.; Felner, A.; Koenig, S.; Ma, H.; Walker, T.; Li, J.; Atzmon, D.; Cohen, L.; Kumar, T. K. S.; Boyarski, E.; and Bartak, R. 2019a. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. arXiv:1906.08291.
- Stern, R.; Sturtevant, N. R.; Felner, A.; Koenig, S.; Ma, H.; Walker, T. T.; Li, J.; Atzmon, D.; Cohen, L.; Kumar, T. K. S.; Boyarski, E.; and Bartak, R. 2019b. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. *Symposium on Combinatorial Search (SoCS)*, 151–158.